

20 de octubre de 2025

Herramientas básicas de
seguridad

Asignatura:
Seguridad y alta disponibilidad

Diego Vargas Gómez de Requena

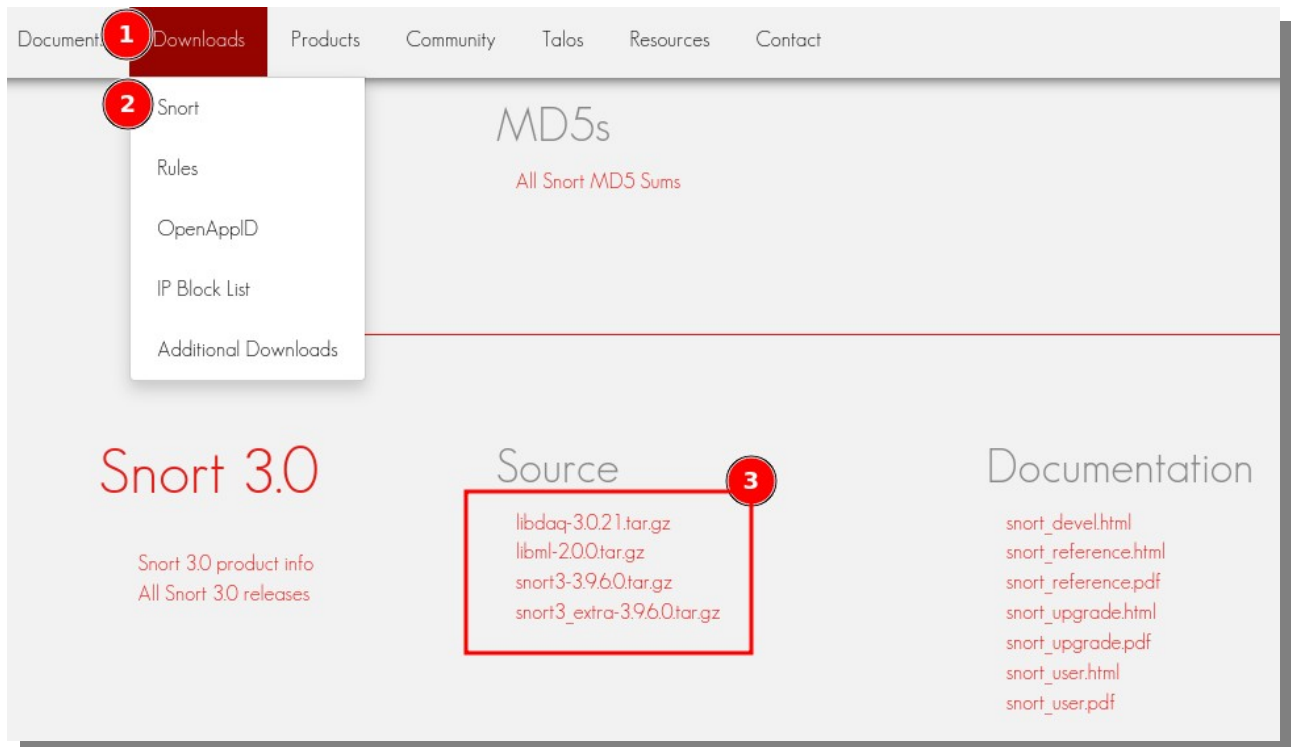


Índice

Instalación Snort3 en Debian11.....	3
Instalación de dependencias.....	4
Compilación e instalación de libdaq.....	5
Compilación e instalación de libml.....	5
Compilación e instalación de Snort3.....	6
Compilación e instalación de Snort3 Extra.....	7
Configuración Inicial.....	9
Explicación carpetas.....	11
Configuración.....	15
Reglas.....	26
Elementos principales de una regla.....	27
Reglas por defecto.....	31
Reglas de la comunidad.....	33
Reglas propias.....	35
Prueba de funcionamiento.....	37
Explicación uso Snort3.....	37
Configuración Snort3 para comprobación.....	40
Comprobación al generar tráfico.....	43
Instalación y configuración del servidor de correos.....	44
Creación del servicio (Postfix + Snort3).....	49
Comprobación del servicio.....	52

Instalación Snort3 en Debian11

Lo primero que haré será descargar los archivos necesarios para la compilación e instalación de Snort3, para ello voy a la [web oficial](#) y descargaré los archivos que se ven en la siguiente captura:



Nota: Esta es una manera de descargar los componentes para la instalación de Snort3 pero yo voy a clonar los repositorios oficiales, por lo que el paso anterior me lo voy a saltar.

Este software tiene candidato de instalación con apt pero en versiones más antiguas, y la instalación la voy a realizar en Debian11 porque es el más actual que contiene todos los paquetes requisitos en sus repositorios.

Si se quisiera instalar en un Debian más actual se debería de añadir al sources.list los repositorios de la versión Bullseye (Debian11) ya que paquetes como libpcrc3-dev no están disponibles.



Voy a explicar un poco que es importante instalar y que no de los archivos anteriormente descargados:

Componente	Obligatorio	Función
libdaq	Sí	Captura de tráfico desde la red
libml	No	Funciones de machine learning
snort3	Sí	Motor IDS/IPS principal
snort3_extra	No	Scripts y módulos adicionales

Instalación de dependencias

Ahora voy a pasar con la instalación de las dependencias que son las siguientes:

```
sudo apt install build-essential cmake libpcap-dev libpcr3-dev  
libdumbnet-dev bison flex zlib1g-dev liblua5.1-dev libssl-  
dev pkg-config autoconf automake libtool libhwloc-dev libpcr2-  
dev xz-utils liblzma-dev uuid-dev libunwind-dev libnuma-dev -y
```

Esto asegura que tengas todo lo necesario para compilar libdaq, libml, snort3 y snort3_extra.



Compilación e instalación de libdaq

Para la compilación e instalación de libdaq voy a realizar los siguientes comandos:

```
git clone https://github.com/snort3/libdaq.git
cd libdaq/
./bootstrap
./configure
make
sudo make install
cd ../
```

Esto instala libdaq en /usr/local/bin.

Para que el sistema encuentre la librería ejecuto el siguiente comando:

```
sudo ldconfig
```

Compilación e instalación de libml

Para la compilación e instalación de libml voy a realizar los siguientes comandos:

```
git clone https://github.com/snort3/libml.git
cd libml/
./configure.sh
cd build/
cmake ..
make
sudo make install
cd ../../
```



Es recomendable ejecutar de nuevo el siguiente comando para asegurar que el sistema encuentre la librería:

```
sudo ldconfig
```

Compilación e instalación de Snort3

Para la compilación e instalación de Snort3 voy a realizar los siguientes comandos:

```
git clone https://github.com/snort3/snort3.git
cd snort3/
./configure_cmake.sh --prefix=/usr/local/snort
cd build/
make
sudo make install
cd ../../
```

Para que el sistema encuentre el binario voy a añadir la ruta /usr/local/bin al PATH:

```
echo 'export PATH=$PATH:/usr/local/snort/bin' >> ~/.bashrc
source ~/.bashrc
```

Nota: Para poder usar el binario con el usuario root será necesario añadir el PATH también en el .bashrc del usuario root.



Se puede verificar la instalación con el comando:

```
snort -version
```

```
diego@snort:~$ snort --version

''-_*> Snort++ <*-
o" )~ Version 3.9.6.0
' ' '
By Martin Roesch & The Snort Team
http://snort.org/contact#team
Copyright (C) 2014-2025 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using DAQ version 3.0.21
Using LibML version 2.0.0
Using libpcap version 1.10.0 (with TPACKET_V3)
Using LuaJIT version 2.1.0-beta3
Using LZMA version 5.2.5
Using OpenSSL 1.1.1w 11 Sep 2023
Using PCRE2 version 10.36 2020-12-04
Using ZLIB version 1.2.11

diego@snort:~$ |
```

Compilación e instalación de Snort3 Extra

Para compilar Snort3 Extra lo primero será añadir la ruta donde se encuentra el fichero snort.pc, que es un fichero de configuración para pkg-config, que es una herramienta que ayuda a compilar software que depende de otras librerías:

```
echo 'export PKG_CONFIG_PATH=/usr/local/snort/lib/pkgconfig:
$PKG_CONFIG_PATH' >> ~/.bashrc
source ~/.bashrc
```

Con el siguiente comando verifico que pkg-config encuentre el fichero en la nueva ruta asignada a la variable PKG_CONFIG_PATH:

```
pkg-config --modversion snort
```



Ahora pasaré con la compilación e instalación de Snort3 Extra, para ello voy a realizar los siguientes comandos:

```
git clone https://github.com/snort3/snort3\_extra.git
cd snort3_extra/
./configure_cmake.sh --prefix=/usr/local/snort
cd build/
make
sudo make install
cd ../../
```

Esto integrará las reglas y scripts extra a la instalación de Snort3.



Configuración Inicial

Ahora voy a empezar a configurar la herramienta, pero antes voy a explicar la estructura de carpetas.

Mi instalación la he realizado en `/usr/local/snort/` por lo que todo el programa cuelga de ahí.



```
/usr/local/snort/
├─ bin
│   ├── appid_detector_builder.sh
│   ├── snort
│   ├── snort2lua
│   ├── u2boat
│   └─ u2spewfoo
├─ etc
│   └─ snort
│       ├── balanced.lua
│       ├── connectivity.lua
│       ├── file_magic.rules
│       ├── inline.lua
│       ├── max_detect.lua
│       ├── security.lua
│       ├── sensitive_data.rules
│       ├── snort_defaults.lua
│       ├── snort.lua
│       └─ talos.lua
├─ include
│   └─ snort
├─ lib
│   ├── pkgconfig
│   │   └─ snort.pc
│   └─ snort
└─ share
    └─ doc
        └─ snort
```



Explicación carpetas

- **/usr/local/snort/bin/** → Contiene los binarios y herramientas con los que se ejecuta y administra Snort, en otras palabras son los programas que se ejecutan desde la terminal.
- **snort** → Binario principal del software.
- **snort2lua** → Herramienta que convierte archivos de configuración de Snort2 como snort.conf que es la versión antigua al nuevo formato Lua que es snort.lua, esto es útil si se migra de una versión a otra.
- **u2boat** → Convierte archivos de registro del formato Unified2 (binario) a formatos legibles como CSV o JSON, esto se usa para los logs.
- **u2spewfoo** → Es similar a u2boat pero este lo muestra por pantalla, en vez de pasar archivos Unified2 a CSV o JSON lo muestra en formato legible por pantalla.
- **appid_detector_builder.sh** → Script que se usa para crear detectores personalizados de aplicaciones, es decir reconocer tráfico de aplicaciones específicas.



- **/usr/local/snort/etc/snort/** → Puede ser una de las carpetas más importantes, contiene los archivos de configuración, reglas y políticas de detección.
- **snort.lua** → Archivo de configuración principal de Snort3, define las redes protegidas (HOME_NET), módulos activos, rutas de reglas, opciones de salida (logs, alertas) y parámetros globales. Es el archivo que se indica con -c al ejecutar Snort3.
- **snort_defaults.lua** → Archivo donde se definen los valores y rutas por defecto del sistema, por ejemplo donde están las reglas.
- **balanced.lua, security.lua, connectivity.lua, max_detect.lua, talos.lua** → Son plantillas de políticas de detección que ya vienen configuradas. Cada una ajusta la sensibilidad del IDS según las necesidades:
 - **balanced.lua** → Equilibrio entre rendimiento y detección.
 - **security.lua** → Prioriza la detección exhaustiva (más recursos).
 - **connectivity.lua** → Prioriza rendimiento, menos reglas activas.
 - **max_detect.lua** → Máxima detección posible.
 - **talos.lua** → Política mantenida por el equipo de Cisco Talos.
- **file_magic.rules** → Reglas para identificación de archivos según sus cabeceras (file magic numbers).
- **sensitive_data.rules** → Reglas para detección de datos sensibles (contraseñas, tarjetas, etc).
- **inline.lua** → Configuración especial para usar Snort3 en modo IPS (Inline), es cuando intercepta y bloquea tráfico en tiempo real.



- **/usr/local/snort/include/snort/** → Contiene archivos de cabecera (.h) que forman parte del código fuente de Snort3. No son necesarios para usar el software pero si para desarrollar o compilar módulos adicionales.
- **/usr/local/snort/lib/pkgconfig/snort.pc** → Es un descriptor usado por pkg-config que es una herramienta estándar de Linux para localizar librerías y dependencias en compilaciones. Sirve para poder compilar e instalar el software completo sin errores.
- **/usr/local/snort/lib/snort/** → Contiene bibliotecas dinámicas (.so) que Snort3 carga al inicio para activar sus funcionalidades. Se usan para implementar módulos, inspectores y decodificadores que analizan protocolos de red.
- **/usr/local/snort/share/doc/snort/** → Contiene documentación oficial y ejemplos para el software.



Carpeta	Función principal	Qué contiene
/usr/local/snort/bin/	Ejecución	Programas y utilidades del IDS
/usr/local/snort/etc/snort/	Configuración	Archivos .lua y reglas .rules
/usr/local/snort/include/snort/	Desarrollo	Cabeceras del motor (C/C++)
/usr/local/snort/lib/pkgconfig/snort.pc	Integración	Descriptor de librerías para pkg-config
/usr/local/snort/lib/snort/	Funcionalidad interna	Librerías dinámicas y módulos del motor
/usr/local/snort/share/doc/snort/	Documentación	Manuales, ejemplos, licencias



Configuración

Antes de configurar Snort3 voy a aclarar una cosa.

Cuando en un entorno real se quiere usar un sistema IDS/IPS como Snort3, se necesita conectar el equipo que contiene el software a un port mirroring del switch final de la infraestructura, esto para que le llegue todo el tráfico de la red al equipo que contiene el IDS/IPS.

Como yo voy a usar una máquina virtual en la red default de QEMU/KVM (192.168.122.0/24) para hacer las pruebas, voy a instalar en el mismo equipo que tengo Snort3 los servicios (como por ejemplo apache), esto para que todo el tráfico le llegue al IDS/IPS.

Aclarado esto voy a explicar la estructura del fichero de configuración principal snort.lua.

Este archivo se organiza en 8 secciones principales:



- **1. configure defaults** → Sin esta configuración Snort3 no sabrá que tráfico inspeccionar ni qué reglas aplicar. En este bloque se definen tres configuraciones:
 - **HOME_NET = 'any'** → Variable que define la red a proteger, por defecto any, que hace referencia a cualquiera, pero en mi caso será la 192.168.122.0/24.
 - **EXTERNAL_NET = 'any'** → Variable que define las redes externas, por defecto any, pero en mi caso será !\$HOME_NET. Esto indica todo excepto mi red.
 - **include 'snort_defaults.lua'** → Carga los valores por defecto de las rutas, variables y módulos.

```
-- 1. configure defaults
--
-- HOME_NET and EXTERNAL_NET must be set now
-- setup the network addresses you are protecting
HOME_NET = 'any'
--
-- set up the external network addresses.
-- (leave as "any" in most situations)
EXTERNAL_NET = 'any'
include 'snort_defaults.lua'
```



- **2. configure inspection** → Esta sección es el motor de análisis de Snort3.
 - Aquí se activan los módulos de inspección que analizan el tráfico.
 - **Stream modules** → Reensamblan flujos TCP/UDP/ICMP.
 - **Protocol modules** → Inspeccionan protocolos específicos como DNS, HTTP, FTP, SIP, SSL, etc.
 - **File ID** → Identifica archivos por cabecera (file_magic.rules).
 - **appid** → Esta configuración es opcional y permite detectar aplicaciones y protocolos avanzados.
 - Algunos módulos requieren configuración adicional, como reputation (listas negras/blancas).

```
-- 2. configure inspection
--
-- mod = { } uses internal defaults
-- you can see them with snort --help-module mod
--
-- mod = default_mod uses external defaults
-- you can see them in snort_defaults.lua
--
-- the following are quite capable with defaults:

stream = { }
stream_ip = { }
stream_icmp = { }
stream_tcp = { }
stream_udp = { }
stream_user = { }
stream_file = { }
```



```
-- see snort_defaults.lua for default_*
gtp_inspect = default_gtp
port_scan = default_med_port_scan
smtp = default_smtp

ftp_server = default_ftp_server
ftp_client = { }
ftp_data = { }

http_inspect = { }
http2_inspect = { }

-- see file_magic.rules for file id rules
file_id = { rules_file = 'file_magic.rules' }
file_policy = { }

js_norm = default_js_norm

-- the following require additional configuration to be fully effective:
appid =
{
    -- appid requires this to use appids in rules
    --app_detector_dir = 'directory to load appid detectors from'
}

--[[
reputation =
{
    -- configure one or both of these, then uncomment reputation
    -- (see also related path vars at the top of snort_defaults.lua)

    --blacklist = 'blacklist file name with ip lists'
    --whitelist = 'whitelist file name with ip lists'
}
--]]
```

- **3. configure bindings** → Es la manera en que Snort3 sabe qué analizador aplicar a cada flujo de red.
 - Indica qué módulo inspecciona qué puerto o servicio. Por ejemplo:
 - TCP 53 → módulo DNS
 - TCP 502 → módulo Modbus
 - TCP 80 → módulo HTTP
 - El wizard permite detectar automáticamente servicios comunes sin definir todos los puertos.

```
-----
-- 3. configure bindings
-----

wizard = default_wizard

binder =
{
  -- port bindings required for protocols without wizard support
  { when = { proto = 'udp', ports = '53', role='server' }, use = { type = 'dns' } },
  { when = { proto = 'tcp', ports = '53', role='server' }, use = { type = 'dns' } },
  { when = { proto = 'tcp', ports = '111', role='server' }, use = { type = 'rpc_decode' } },
  { when = { proto = 'tcp', ports = '502', role='server' }, use = { type = 'modbus' } },
  { when = { proto = 'tcp', ports = '2123 2152 3386', role='server' }, use = { type = 'gtp_inspect' } },
  { when = { proto = 'tcp', ports = '2484', role='server' }, use = { type = 'iec104' } },
  { when = { proto = 'udp', ports = '2222', role = 'server' }, use = { type = 'cip' } },
  { when = { proto = 'tcp', ports = '44818', role = 'server' }, use = { type = 'cip' } },
}
```

- **4. configure performance** → Esta configuración es útil para redes con mucho tráfico.
 - Permite medir y optimizar el rendimiento del IDS.
 - **latency** → Monitoriza retrasos en la inspección.
 - **profiler** y **perf_monitor** → Generan estadísticas internas de uso de CPU, memoria y paquetes.

```
-----
-- 4. configure performance
-----

-- use latency to monitor / enforce packet and rule thresholds
--latency = { }

-- use these to capture perf data for analysis and tuning
--profiler = { }
--perf_monitor = { }
```



- **5. configure detection** → Aquí es donde Snort3 decide qué tráfico genera alerta.
- **references** → Mapas de referencias externas (CVE, BID, CERT, etc).
- **classifications** → Define categorías de alertas (malware, DoS, etc).
- **ips** → Contiene las reglas y variables utilizadas por Snort3 para generar alertas.
- Se pueden habilitar reglas integradas (`enable_builtin_rules = true`) o cargar reglas externas (`include /ruta/rules`).

```
-----  
-- 5. configure detection  
-----  
  
references = default_references  
classifications = default_classifications  
  
ips =  
{  
    -- use this to enable decoder and inspector alerts  
    --enable_builtin_rules = true,  
  
    -- use include for rules files; be sure to set your path  
    -- note that rules files can include other rules files  
    -- (see also related path vars at the top of snort_defaults.lua)  
  
    variables = default_variables  
}  
  
-- use these to configure additional rule actions  
-- react = { }  
-- reject = { }  
  
-- use this to enable payload injection utility  
-- payload_injector = { }
```



- **6. configure filters** → Este bloque de configuración es útil para evitar falsos positivos o reducir spam de alertas en producción.
 - Permite controlar qué alertas se registran y cómo:
 - `suppress` → Bloquea alertas de reglas específicas.
 - `event_filter` → Limita cantidad de eventos para reglas concretas.
 - `rate_filter` → Genera alertas solo si se supera un umbral de conexiones/paquetes.

```
-----
-- 6. configure filters
-----

-- below are examples of filters
-- each table is a list of records

--[[
suppress =
{
    -- don't want to any of see these
    { gid = 1, sid = 1 },

    -- don't want to see anything for a given host
    { track = 'by_dst', ip = '1.2.3.4' },

    -- don't want to see these for a given host
    { gid = 1, sid = 2, track = 'by_dst', ip = '1.2.3.4' },
}
--]]

--[[
event_filter =
{
    -- reduce the number of events logged for some rules
    { gid = 1, sid = 1, type = 'limit', track = 'by_src', count = 2, seconds = 10 },
    { gid = 1, sid = 2, type = 'both', track = 'by_dst', count = 5, seconds = 60 },
}
--]]

--[[
rate_filter =
{
    -- alert on connection attempts from clients in SOME_NET
    { gid = 135, sid = 1, track = 'by_src', count = 5, seconds = 1,
      new_action = 'alert', timeout = 4, apply_to = '[$SOME_NET]' },

    -- alert on connections to servers over threshold
    { gid = 135, sid = 2, track = 'by_dst', count = 29, seconds = 3,
      new_action = 'alert', timeout = 1 },
}
--]]
```



- **7. configure outputs** → Aquí es donde se configuran las salidas de los comandos de Snort3.
 - Define cómo y dónde Snort3 genera alertas o logs:
 - CSV, formato rápido, formato completo, syslog, Unified2 (estándar para herramientas de análisis).
 - También puede loggear paquetes completos (log_pcap, log_hext) o escribir en archivos (file_log).
 - Se pueden habilitar desde línea de comandos (-A para alertas, -L para logs de paquetes) o aquí de forma permanente.

```
-- 7. configure outputs
--
-- event logging
-- you can enable with defaults from the command line with -A <alert_type>
-- uncomment below to set non-default configs
--alert_csv = { }
--alert_fast = { }
--alert_full = { }
--alert_sfsocket = { }
--alert_syslog = { }
--unified2 = { }

-- packet logging
-- you can enable with defaults from the command line with -L <log_type>
--log_codecs = { }
--log_hext = { }
--log_pcap = { }

-- additional logs
--packet_capture = { }
--file_log = { }
```



- **8. configure tweaks** → Aquí es donde se incluyen archivos de configuración adicionales.
 - Permite incluir un archivo Lua adicional con ajustes personalizados.
 - Útil para configuraciones avanzadas sin tocar el snort.lua principal.

```
-- 8. configure tweaks
--
if ( tweaks ~= nil ) then
    include(tweaks .. '.lua')
end
```



Configuración	Función
Defaults	Define redes y valores base
Inspection	Módulos y decodificadores que analizan tráfico
Bindings	Asigna módulos a puertos o servicios
Performance	Métricas y optimización de velocidad
Detection	Reglas y alertas, referencias externas
Filters	Control de alertas, limitación de spam
Outputs	Define cómo se registran logs y alertas
Tweaks	Ajustes adicionales personalizados



Una vez explicado el fichero principal de configuración de Snort3 voy a configurar lo principal para empezar a usarlo que son las variables del apartado configure defaults:

```
-- 1. configure defaults
--
-- HOME_NET and EXTERNAL_NET must be set now
-- setup the network addresses you are protecting
HOME_NET = '192.168.122.0/24'
-- set up the external network addresses.
-- (leave as "any" in most situations)
EXTERNAL_NET = '!$HOME_NET'
include 'snort_defaults.lua'
```

Como se muestra en la imagen anterior en la variable de HOME_NET he añadido la red que quiero proteger, y en la variable EXTERNAL_NET he definido con !, todo lo que no sea mi red a proteger.



Reglas

Las reglas de Snort3 se componen de la siguiente estructura:

```
acción  protocolo  IP_origen  puerto_origen  ->  IP_destino  
puerto_destino (opciones)
```

Nota: Esta es una estructura básica de las reglas de Snort3.

Por ejemplo la siguiente:

```
alert  tcp  any  any  ->  any  80  (msg:"HTTP  GET  detected";  
sid:1000001; rev:1;)
```

Esta regla alerta de todas las peticiones que se hagan a un puerto 80.

Lo que se dice con esta regla es: peticiones que se hagan desde cualquier IP con cualquier puerto, hacia (->) cualquier IP con puerto 80. Muestra el mensaje HTTP GET detected.

Elementos principales de una regla

- **Acción** → Qué hacer cuando se cumple la condición.
- **Protocolo** → Tipo de tráfico que se filtra (TCP, UDP, ICMP o IP).
- **IP origen / destino** → Direcciones IP que se comparan.
- **Puerto origen / destino** → Puerto/s a filtrar.
- **Opciones** → Condiciones adicionales, mensajes, referencias, etc.



Action	Protocol	Source IP	Source Port	Direction	Destination IP	Destination Port	Options
Alert	TCP	ANY	ANY	<>	ANY	ANY	Msg
Drop	UDP						Reference
Reject	ICMP						Sid
							Rev

Rule Header
Rule Options

The following rule will generate an alert for each ICMP packet processed by snort;

```

alert icmp any any <> any any { msg: "ICMP Packet found"; reference: CVE-XXXX; sid: 1000001; rev: 1; }

```

Rule Header

alert	Action type	
icmp	Protocol type	
any	Source IP	
any	Source port	
<>	Direction	
any	Destination IP	
any	Destination port	

Rule Options

msg	Message	
reference	Reference	
sid	Rule id	
rev	Revision information	



Acciones

- **alert** → Genera una alerta y registra el paquete. (modo IDS)
- **log** → Solo registra el paquete.
- **drop** → Bloquea y registra el paquete. (modo IPS)
- **reject** → Bloquea, registra y cierra la conexión. (modo IPS)

Snort3 funciona en modo IDS (detección) por defecto. Para convertirlo en IPS (prevención), hay que activar el modo inline.

Protocolo

Snort3 solo reconoce IP, TCP, UDP e ICMP.

Para detectar tráfico de aplicaciones (HTTP, FTP, etc.), debes filtrar por puerto.

Ejemplo: detectar FTP (puerto 21)

```
alert tcp any any -> any 21 (msg:"FTP traffic detected";  
sid:1000002; rev:1;)
```

Ips y Puertos

Snort3 permite filtrar Ips, rangos, excluir direcciones y filtrar por puertos o rangos.

Ejemplos de IP:

- **IP específica** → `alert icmp 192.168.122.56...`
- **Rango / Subred** → `alert icmp 192.168.122.0/24...`
- **Múltiples rangos** → `alert icmp [192.168.122.0/24,10.0.0.0/22]...`
- **Exclusión** → `alert icmp !192.168.122.0/24...`

Ejemplos de Puertos:

- **Puerto específico** → `alert tcp any 21...`
- **Excluir puerto** → `alert tcp any !21...`
- **Rango (1-1024)** → `alert tcp any 1:1024...`
- **Menor o igual a 1024** → `alert tcp any :1024...`
- **Mayor o igual a 1025** → `alert tcp any 1025:...`



- **Múltiples puertos** → alert tcp any [21,23]...

Dirección del flujo

- -> De origen a destino.
- <> Bidireccional.

Tipos de opciones de reglas

- **General Rule Options** → Información básica (msg, sid, rev, reference).
- **Payload Rule Options** → Analizan el contenido del paquete.
- **Non-Payload Rule Options** → Analizan metadatos del paquete (flags, tamaño, etc).

Opciones Generales

- **msg** → Mensaje que se muestra cuando la regla se activa.
- **sid** → ID único de la regla:
 - <100 → reservados
 - 100-999999 → reglas del sistema
 - >=1000000 → reglas creadas por el usuario
- **reference** → Referencia (por ejemplo, CVE o URL).
- **rev** → Número de revisión de la regla. (útil para control de versiones manual).

Ejemplo:

```
alert icmp any any -> any any (msg:"ICMP Packet Found";  
sid:1000003; reference:cve,CVE-XXXX; rev:1;)
```

Opciones de Detección de Payload

- **content** → Busca texto o bytes específicos (ASCII o HEX).
- **nocase** → Ignora mayúsculas/minúsculas.
- **fast_pattern** → Prioriza una búsqueda específica (optimiza rendimiento).



Ejemplo:

```
alert tcp any any -> any 80 (msg:"GET Request Found";  
content:"GET"; fast_pattern; content:"www"; sid:1000004; rev:1;)
```

Opciones de Detección No-Payload

- **id** → Filtra por ID de paquete IP.
- **flags** → Filtra por banderas TCP (SYN, ACK, FIN, etc).
- **dsize** → Filtra por tamaño de payload.
- **sameip** → Detecta si IP origen = IP destino.



Reglas por defecto

Snort3 tiene 3 maneras de usar reglas:

- **Community Rules** → Reglas gratis. Son públicas y no requieren registro.
- **Registered Rules** → Reglas gratis pero que requieren registro en la web de Snort. Son las mismas reglas que las de pago pero con 30 días de retraso.
- **Subscriber Rules** → Reglas de pago que se actualizan dos veces por semana (martes y jueves). Requieren suscripción activa.

Snort3 viene con reglas por defecto como las configuradas en el archivo `/usr/local/snort/etc/snort/sensitive_data.rules`.

A continuación voy a elegir una regla de este archivo para explicarla:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET any ( msg:"SENSITIVE-  
DATA Email Addresses"; http_uri; sd_pattern:"email", threshold  
20; service:http; classtype:sdf; gid:13; sid:18; rev:1; )
```

- **Opciones generales:**
 - **alert** → Acción: genera una alerta cuando se cumple la condición.
 - **tcp** → Protocolo sobre el que se aplica.
 - **\$HOME_NET any** → Origen: cualquier puerto dentro de la red protegida.
 - **->** → Dirección del tráfico: desde el cliente hacia el servidor.
 - **\$EXTERNAL_NET any** → Destino: cualquier puerto fuera de la red protegida.

Nota: Detecta cuando desde mi red se envían correos o datos personales hacia fuera vía HTTP.



- **Opciones específicas:**

- **msg** → Mensaje que se mostrará en la alerta.
- **http_uri** → Indica que la inspección se hará sobre la URI HTTP (la parte del enlace que el cliente solicita).
- **sd_pattern, threshold 20** → significa "Sensitive Data Pattern" y sirve para buscar un patrón predefinido, en este caso "email". El threshold 20 indica que solo lanzará una alerta si detecta 20 o más coincidencias en el mismo flujo de datos.
- **service** → Aplica solo a tráfico HTTP (nivel aplicación).
- **classtype** → Clasificación del tipo de alerta. sdf detecta fuga de datos sensibles.
- **gid** → "Generator ID", indica el módulo que generó la alerta. El 13 corresponde con el inspector de datos sensibles.
- **sid** → "Snort ID", identificador único de la regla dentro del conjunto.
- **rev** → Revisión de la regla, útil por si se actualiza en un futuro.

Nota: En resumen esta regla genera una alerta si detecta en tráfico HTTP saliente desde mi red al menos 20 direcciones de correo electrónico dentro de la URI del HTTP.

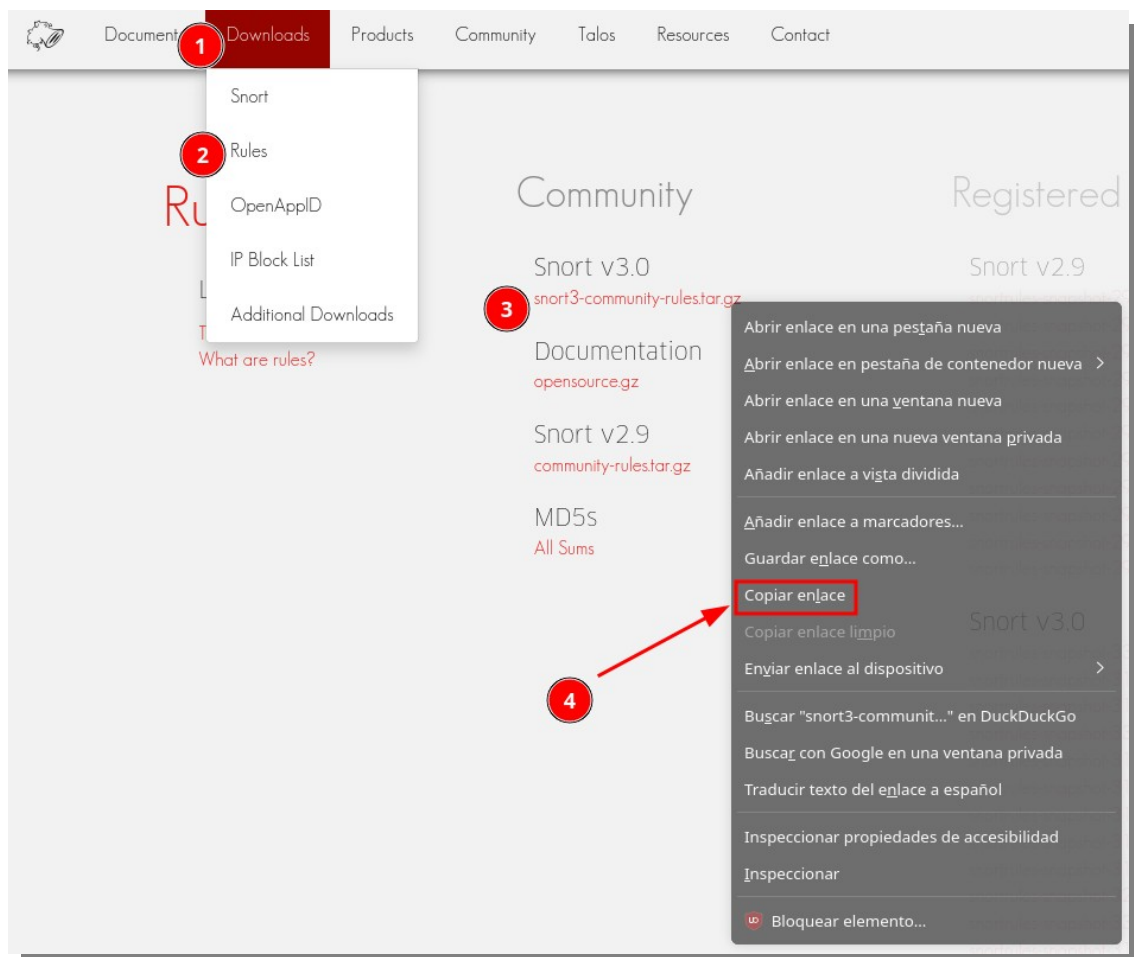
Reglas de la comunidad

A continuación voy a descargar y configurar las reglas de la comunidad.

Lo primero que haré será crear la carpeta rules ya que no se crea por defecto, en esta carpeta vamos a guardar las reglas que descarguemos y las que creemos nosotros.

```
mkdir /usr/local/snort/etc/rules
```

Una vez creada la carpeta voy a descargar el archivo de la comunidad con todas las reglas, para ello copiaremos el link de la web oficial de Snort como se muestra en la siguiente captura:





Y con el comando `wget` y `tar` descargare el comprimido y lo descomprimiré:

```
cd /usr/local/snort/etc/rules/  
wget https://www.snort.org/downloads/community/snort3-community-rules.tar.gz  
tar xfvz snort3-community-rules.tar.gz  
mv snort3-community-rules/snort3-community.rules .  
rm -r snort3-community-rules*
```

Para que Snort3 use esas reglas será necesario indicarlo en el archivo principal de configuración (`snort.lua`):

```
ips =  
{  
  -- use this to enable decoder and inspector alerts  
  --enable_builtin_rules = true,  
  
  -- use include for rules files; be sure to set your path  
  -- note that rules files can include other rules files  
  -- (see also related path vars at the top of snort_defaults.lua)  
  
  variables = default_variables,  
  rules = [[  
    include /usr/local/snort/etc/rules/snort3-community.rules  
  ]]  
}
```

Nota: Este fichero incluye 4017 reglas actualmente.

```
root@snort:/usr/local/snort/etc/rules# wc -l snort3-community.rules  
4017 snort3-community.rules  
root@snort:/usr/local/snort/etc/rules# |
```

Si se quiere comprobar que carguen correctamente se puede ejecutar el siguiente comando:

```
snort -c /usr/local/snort/etc/snort/snort.lua -T
```

Nota: Con el parámetro `-T` se hace una comprobación de la configuración.



Y analizando la salida se puede verificar que se carguen todas las reglas:

```
-----  
ips policies rule stats  
      id  loaded  shared enabled  file  
      0    4236      0    4236  /usr/local/snort/etc/snort/snort.lua  
-----
```

Reglas propias

Ahora como última configuración de reglas voy a configurar algunas reglas propias para probar el funcionamiento del IDS/IPS posteriormente.

Normalmente se suelen configurar en el archivo `/usr/local/snort/etc/rules/local.rules`

```
# Detectar ping de la red de fuera hacia mi red  
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"Ping de fuera de la red detectado"; sid:1000001; rev:1;)  
  
# Detectar intento de conexión SSH de fuera hacia mi red  
alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"Intento de conexión SSH detectado"; sid:1000002; rev:1;)  
  
# Detectar petición HTTP de fuera hacia mi red  
alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (msg:"Petición HTTP detectada"; content:"GET", nocase; sid:1000003; rev:1;)
```

Para configurar reglas más complejas hay herramientas en línea como las del siguiente [enlace](#).

Para que Snort3 las cargue se tienen que configurar en el archivo de configuración principal `snort.lua`:

```
ips =  
{  
  -- use this to enable decoder and inspector alerts  
  --enable_builtin_rules = true,  
  
  -- use include for rules files; be sure to set your path  
  -- note that rules files can include other rules files  
  -- (see also related path vars at the top of snort_defaults.lua)  
  
  variables = default_variables,  
  rules = {  
    include /usr/local/snort/etc/rules/local.rules  
  }  
}
```



Como se ve en la captura anterior he eliminado la llamada a las reglas de la comunidad ya que no necesito cargar tantas reglas. Lo habitual al usar un IDS/IPS es configurar las reglas que se necesitan y nada más, ya que si no se vuelve insostenible a largo plazo.



Prueba de funcionamiento

Ahora voy a pasar a probar el funcionamiento de las reglas pero antes voy a explicar los diferentes modos que tiene Snort3 de funcionar y los parámetros a usar aunque Snort3 ha cambiado con respecto a Snort2, ya que ahora los modos ya no se activan con los mismos parámetros. Todo se gestiona con el archivo de configuración snort.lua y algunos parámetros.

Snort3 tiene los siguientes modos de funcionamiento:

Explicación uso Snort3

1. Sniffer / Packet Capture

- Captura paquetes y los muestra en tiempo real.
- Se activa con los siguiente parámetros:

```
snort -i enp1s0 -c /usr/local/snort/etc/snort/snort.lua -v
```

- **-v** → verbose, muestra paquetes básicos.

En Snort3 no existe -A console como si existía en Snort2. Para ver alertas en tiempo real, se habilitan outputs en el fichero de configuración snort.lua (alert_csv, alert_fast, etc) o se ejecuta -v para modo sniffer.

2. Logger / Guardado de paquetes

- Guarda paquetes en archivos, para realizarles análisis posteriormente.
- Se configura en el bloque outputs del fichero snort.lua.

```
log_pcap = {}
```

- Se ejecuta de la siguiente manera:

```
snort -i enp1s0 -c /usr/local/snort/etc/snort/snort.lua -L  
log_pcap -l /var/log/snort
```

Con `-L log_pcap + -l /ruta/` se define el tipo de log y dónde se guarda.

Nota: La carpeta /var/log/snort/ no se crea por defecto al instalar el software por lo que se tendrá que crear manualmente.

3. IDS Mode (detección de intrusos)

- Aplica reglas y genera alertas según lo definido en snort.lua y reglas incluidas (local.rules, snort3_extra, etc).
- Se debe configurar snort.lua para añadir las reglas deseadas como hice anteriormente con la configuración de la siguiente captura:

```
ips =  
{  
  -- use this to enable decoder and inspector alerts  
  --enable_builtin_rules = true,  
  
  -- use include for rules files; be sure to set your path  
  -- note that rules files can include other rules files  
  -- (see also related path vars at the top of snort_defaults.lua)  
  
  variables = default_variables,  
  rules = [  
    include /usr/local/snort/etc/rules/local.rules  
  ]  
}
```



- Se ejecuta con la siguiente línea:

```
snort -i enp1s0 -c /usr/local/snort/etc/snort/snort.lua
```

- Las alertas se manejan según el bloque outputs en el fichero snort.lua:

```
alert_fast = {}  
alert_full = {}  
alert_csv = {}  
alert_syslog = {}  
unified2 = {}
```

4. IPS Mode (inline)

- Bloquea tráfico según las reglas configuradas.
- Los parámetros son los siguientes

```
-Q -daq <tipo> --daq-var <var>
```

- Ejemplo con NFQUEUE:

```
snort -Q -daq nfqueue -daq-var queue=0 -i enp1s0 -c  
/usr/local/snort/etc/snort/snort.lua
```

Nota: El comportamiento depende del DAQ que se use, hay que configurar el sistema para que el tráfico pase por Snort3 (iptables + NFQUEUE, bridge, etc).

5. Pcap Processing (offline)

- Analiza archivos .pcap previamente capturados.
- Los parámetros son:

```
-r <archivo.pcap>
```

- Un ejemplo sería:

```
snort -r captura.pcap -c /usr/local/snort/etc/snort/snort.lua
```



Resumen parámetros

- **-i <iface>** → Interfaz de red
- **-A <tipo>** → Modo de alerta
- **-D** → Ejecuta el software en segundo plano
- **-c <archivo>** → Configuración principal
- **-v** → Verbose, para modo sniffer
- **-L <log_type>** → Tipo de log de paquetes (log_pcap, dump, etc)
- **-l <ruta>** → Directorio para guardar logs
- **-r <archivo>** → Procesar pcap offline
- **-Q** → Inline mode (IPS)
- **--daq <tipo>** → Selecciona DAQ
- **--daq-var <var>** → Configura DAQ inline

Configuración Snort3 para comprobación

Para configurar Snort3 lo primero será identificar como quieres usarlo, yo lo que quiero es usarlo en modo IDS que es su comportamiento básico, pero además quiero también que guarde los logs.

1. Ya configuramos anteriormente las variables de red:

```
-----  
-- 1. configure defaults  
-----  
  
-- HOME_NET and EXTERNAL_NET must be set now  
-- setup the network addresses you are protecting  
HOME_NET = '192.168.122.0/24'  
  
-- set up the external network addresses.  
-- (leave as "any" in most situations)  
EXTERNAL_NET = '!$HOME_NET'  
  
include 'snort_defaults.lua'
```



2. También configuramos las reglas que queremos que use:

```
ips =
{
  -- use this to enable decoder and inspector alerts
  --enable_builtin_rules = true,

  -- use include for rules files; be sure to set your path
  -- note that rules files can include other rules files
  -- (see also related path vars at the top of snort_defaults.lua)

  variables = default variables,
  rules = [[
    include /usr/local/snort/etc/rules/local.rules
  ]]
}
```

```
# Detectar ping de la red de fuera hacia mi red
alert icmp any any -> $HOME_NET any (msg:"Ping de fuera de la red detectado"; sid:1000001; rev:1;)

# Detectar intento de conexión SSH de fuera hacia mi red
alert tcp any any -> $HOME_NET 22 (msg:"Intento de conexión SSH detectado"; sid:1000002; rev:1;)

# Detectar petición HTTP de fuera hacia mi red
alert tcp any any -> $HOME_NET 80 (msg:"Petición HTTP detectada"; content:"GET", nocase; sid:1000003; rev:1;)
```

Nota: Las reglas las he modificado ya que me he dado cuenta que al configurar que detecte todo el tráfico que llegue de fuera, hacia la red que quiero proteger no iba a detectar nada, esto porque mi máquina con Snort3 está en la red 192.168.122.0/24 y la máquina desde donde mando las peticiones está en la misma red (192.168.122.1, máquina host). La regla la he modificado para que la IP de origen no solo sea de fuera de mi red, si no todas.

3. Ahora lo que configuraré será los outputs que quiero que tenga el software:

```
-----  
-- 7. configure outputs  
-----  
  
-- event logging  
-- you can enable with defaults from the command line with -A <alert_type>  
-- uncomment below to set non-default configs  
alert_csv = { file = true }  
alert_fast = { file = true }  
alert_full = { file = true }  
--alert_ssocket = { }  
--alert_syslog = { }  
--unified2 = { }  
  
-- packet logging  
-- you can enable with defaults from the command line with -L <log_type>  
--log_codecs = { }  
--log_hext = { }  
--log_pcap = { }  
  
-- additional logs  
--packet_capture = { }  
--file_log = { }
```

Esto hará lo siguiente:

- **alert_fast** → alertas rápidas en formato simple
- **alert_full** → alertas detalladas
- **alert_csv** → alertas en formato CSV

La opción **file = true** se usa para que genere archivos de log.

Habrá que crear la carpeta `/var/log/snort/` para que se guarden ahí los logs. Esto hay que hacerlo manualmente porque en la instalación de Snort3 no se hace automáticamente:

```
mkdir /var/log/snort/
```

Si se quiere ver desde la consola se puede usar el parámetro `-v`:

```
snort -i enp1s0 -c /usr/local/snort/etc/snort/snort.lua -v
```



4. Ya podré ejecutar Snort3 y verificar las reglas configuradas:

Para lograrlo voy a ejecutar el siguiente comando:

```
snort -i enp1s0 -c /usr/local/snort/etc/snort/snort.lua -l /var/log/snort/
```

Este comando activa el IDS por defecto:

- Escucha tráfico en la interfaz enp1s0.
- Aplica la configuración realizada en snort.lua.
- Guarda los outputs configurados en /var/log/snort/

Nota: Con este comando al no indicar el modo de alerta (-A) se guarda la salida en los diferentes logs que se han configurado anteriormente en snort.lua (fast, full y csv).

Comprobación al generar tráfico

Voy a ejecutar el IDS y voy a generar el tráfico que esperan las alertas que configuraré manualmente antes.

```
snort -i enp1s0 -c /usr/local/snort/etc/snort/snort.lua -l /var/log/snort/ -D
```

En Snort3 modo IDS (comportamiento por defecto), no se necesita ningún parámetro especial. Solo asegurarse de:

- Configurar correctamente HOME_NET Y EXTERNAL_NET.
- Incluir las reglas al bloque ips.
- Activar algún alert_* para ver los resultados.
- Ejecutar Snort3 en la interfaz correcta.



Generación del tráfico

```
> ping -c4 192.168.122.27
PING 192.168.122.27 (192.168.122.27) 56(84) bytes of data.
64 bytes from 192.168.122.27: icmp_seq=1 ttl=64 time=0.195 ms
64 bytes from 192.168.122.27: icmp_seq=2 ttl=64 time=0.112 ms
64 bytes from 192.168.122.27: icmp_seq=3 ttl=64 time=0.299 ms
64 bytes from 192.168.122.27: icmp_seq=4 ttl=64 time=0.113 ms

--- 192.168.122.27 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3073ms
rtt min/avg/max/mdev = 0.112/0.179/0.299/0.076 ms
> ssh diego@192.168.122.27
diego@192.168.122.27's password:
Linux snort 5.10.0-36-amd64 #1 SMP Debian 5.10.244-1 (2025-09-29) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Oct 20 00:55:54 2025 from 192.168.122.1
diego@snort:~$ |
```

Comprobación de los logs

```
root@snort:~# cat /var/log/snort/alert fast.txt
10/20-00:56:28.044567 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.1 -> 192.168.122.27
10/20-00:56:28.044620 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.27 -> 192.168.122.1
10/20-00:56:29.069103 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.1 -> 192.168.122.27
10/20-00:56:29.069127 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.27 -> 192.168.122.1
10/20-00:56:30.097266 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.1 -> 192.168.122.27
10/20-00:56:30.097312 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.27 -> 192.168.122.1
10/20-00:56:31.117189 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.1 -> 192.168.122.27
10/20-00:56:31.117212 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.27 -> 192.168.122.1
10/20-00:57:18.227751 [**] [1:1000002:1] "Intento de conexión SSH detectado" [**] [Priority: 0] {TCP} 192.168.122.1:50716 -> 192.168.122.27:22
```

En la captura se puede ver:

- En el bloque seleccionado más grande, como se detecta el tráfico ICMP.
- En el segundo bloque la conexión SSH.

Instalación y configuración del servidor de correos

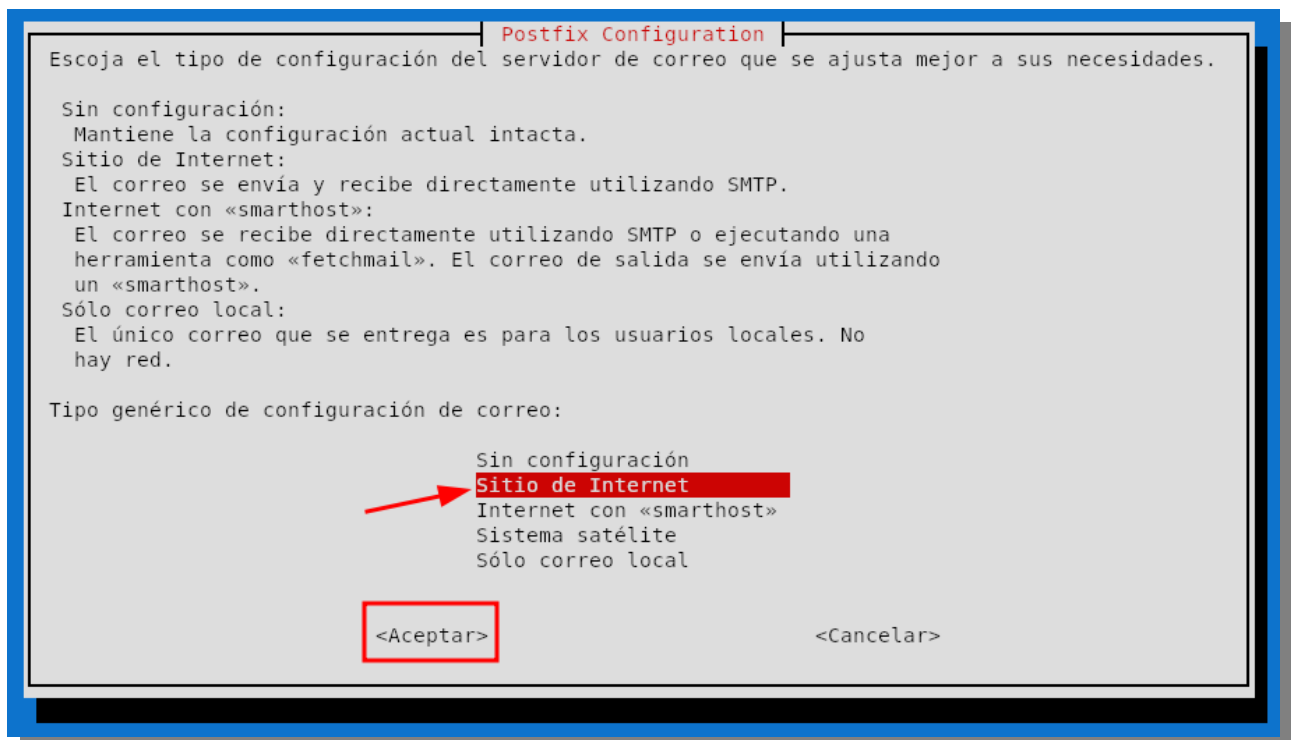
Ahora voy a configurar un servidor de correos como Postfix para que las alertas que detecta el IDS me las envíe a mi correo.

Lo primero será instalar los paquetes necesarios:

```
sudo apt install postfix mailutils -y
```

- **postfix** → Es un servidor de correo.
- **mailutils** → Es un conjunto de herramientas para trabajar con correos desde terminal.

Cuando se instala se pide una configuración inicial:



El «nombre de sistema de correo» es el nombre del dominio que se utiliza para «cu
por favor, no haga que su máquina envíe los correo electrónicos desde root@exampl

Otros programas utilizarán este nombre. Deberá ser un único nombre de dominio cua

Por consiguiente, si una dirección de correo en la máquina local es algo@example.

Nombre del sistema de correo:

snort

<Aceptar>

Ahora como al correo que quiero que me lleguen las alerta es uno de google, será necesario crear una contraseña de aplicación. Esto porque Gmail ya no permite el acceso de aplicaciones menos seguras.





 **Contraseñas de aplicación** 

Resultados de la cuenta de Google

-  **Contraseñas de aplicación**
Seguridad
-  Gestor de contraseñas
Seguridad
-  Notificaciones de inicio de sesión
Seguridad
-  Herramientas de introducción de texto
Información personal

Artículos del Centro de Ayuda

-  Iniciar sesión con contraseñas de aplicación
-  Usar o corregir contraseñas de aplicación

No tienes ninguna contraseña de aplicación.

Para crear una contraseña específica de la aplicación, escribe el nombre de la aplicación a continuación...

Nombre de la aplicación

snort3



Crear



Esto nos genera una contraseña que debemos de guardar en el fichero `/etc/postfix/sasl_passwd`:

```
[smtp.gmail.com]:587 vargasgomezderequenadie@gmail.com:XXXXXXXXXXXXXXXXXXXX
```

A continuación habrá que configurar Postfix en su fichero de configuración principal (`/etc/postfix/main.cf`):

```
#####  
# CONFIGURACIÓN PARA SNORT  
#####  
  
relayhost = [smtp.gmail.com]:587  
  
smtp_use_tls = yes  
smtp_tls_security_level = encrypt  
smtp_tls_note_starttls_offer = yes  
  
smtp_sasl_auth_enable = yes  
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd  
smtp_sasl_security_options = noanonymous  
smtp_sasl_mechanism_filter = plain, login
```

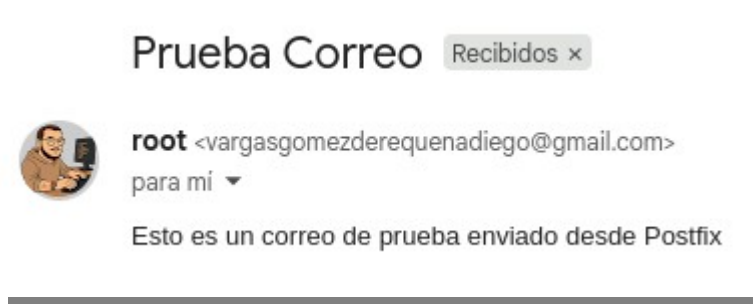
Por último será necesario convertir el archivo de texto plano `/etc/postfix/sasl_passwd` en una base de datos hash que Postfix pueda leer rápidamente y de forma segura, para ello lo hacemos con los siguientes comandos:

```
sudo postmap /etc/postfix/sasl_passwd  
sudo          chmod          600          /etc/postfix/sasl_passwd  
/etc/postfix/sasl_passwd.db
```



Ya estaría todo configurado y podríamos probar a reiniciar el servicio y enviar un correo de prueba:

```
sudo systemctl restart postfix  
echo "Esto es un correo de prueba enviado desde Postfix" | mail  
-s "Prueba Correo" vargasgomezderequenadiego@gmail.com
```



Creación del servicio (Postfix + Snort3)

Lo primero que voy a hacer es crear el script que va a ejecutar el servicio:

```
sudo nano /usr/local/bin/snort-mail.sh
```



```
#!/bin/bash

# Fichero de alertas de Snort
ALERTFILE="/var/log/snort/alert_fast.txt"

# Fichero para acumular alertas
ALERTFILE_BACKUP="/var/log/snort/alert_fast_backup.txt"

# Correo de destino
EMAIL="vargasgomezderequenadiego@gmail.com"

# Intervalo de envío en segundos
INTERVAL=180    # 3 minutos

while true; do

    # Verifica que el fichero $ALERTFILE no esté vacío
    if [ -s "$ALERTFILE" ]; then

        # Guarda las reglas nuevas en $ALERTFILE_BACKUP
        cat "$ALERTFILE" >> "$ALERTFILE_BACKUP"

        # Envía las alertas al $EMAIL
        mail -s "Alerta Snort" "$EMAIL" < "$ALERTFILE"

        # Limpia el fichero original $ALERTFILE
        > "$ALERTFILE"

    fi

    # Espera el tiempo configurado hasta que se vuelva a comprobar el
    # fichero $ALERTFILE
    sleep $INTERVAL

done
```



Ahora le daré permisos de ejecución:

```
sudo chmod +x /usr/local/bin/snort-mail.sh
```

Ya casi terminando crearé el servicio para que se ejecute el script automáticamente al iniciar el equipo, y para poder gestionarlo de una manera más cómoda:

```
sudo nano /etc/systemd/system/snort-mail.service
```

```
[Unit]
Description=Envio de las alertas de Snort por correo
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/snort-mail.sh
Restart=always
User=root

[Install]
WantedBy=multi-user.target
```

Por último voy a cargar el servicio e iniciarlo:

```
sudo systemctl daemon-reload
sudo systemctl enable snort-mail.service
sudo systemctl start snort-mail.service
sudo systemctl status snort-mail.service
```



Comprobación del servicio

Para comprobar el servicio voy a ejecutar Snort3 en segundo plano como daemon, y voy a provocar las alertas:

```
snort -i enp1s0 -c /usr/local/snort/etc/snort/snort.lua -l /var/log/snort/ -D
```

```
root@snort:~# systemctl status snort-mail.service
● snort-mail.service - Envío de las alertas de Snort por correo
   Loaded: loaded (/etc/systemd/system/snort-mail.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2025-10-20 14:48:30 CEST; 2s ago
     Main PID: 1363 (snort-mail.sh)
        Tasks: 2 (limit: 4634)
       Memory: 508.0K
          CPU: 2ms
       CGroup: /system.slice/snort-mail.service
               └─1363 /bin/bash /usr/local/bin/snort-mail.sh
                 └─1364 sleep 180

oct 20 14:48:30 snort systemd[1]: Started Envío de las alertas de Snort por correo.
root@snort:~#
```

```
ping -c4 192.168.122.27
ssh diego@192.168.122.27
```

Alerta Snort Recibidos x



root <vargasgomezderequenadie@gmail.com>
para mí ▾

```
10/20-14:48:49.857913 [**] [1:1000002:1] "Intento de conexión SSH detectado" [**] [Priority: 0] {TCP} 192.168.122.1:46750 -> 192.168.122.27:22
10/20-14:48:55.621169 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.1 -> 192.168.122.27
10/20-14:48:55.621194 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.27 -> 192.168.122.1
10/20-14:48:56.647676 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.1 -> 192.168.122.27
10/20-14:48:56.647720 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.27 -> 192.168.122.1
10/20-14:48:57.671683 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.1 -> 192.168.122.27
10/20-14:48:57.671725 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.27 -> 192.168.122.1
10/20-14:48:58.695670 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.1 -> 192.168.122.27
10/20-14:48:58.695712 [**] [1:1000001:1] "Ping de fuera de la red detectado" [**] [Priority: 0] {ICMP} 192.168.122.27 -> 192.168.122.1
10/20-14:49:11.716696 [**] [1:1000002:1] "Intento de conexión SSH detectado" [**] [Priority: 0] {TCP} 192.168.122.1:46408 -> 192.168.122.27:22
```